**Institut Teknologi Bandung**

Directorate of Sustainable Professional Education

CHAPTER 13

# Timeseries Forecasting

Where a common-sense baseline beats two deep learning models in a row — and what that teaches about architecture priors.

**Duration** 3 hours (2 sessions)

**Instructor** Prof. Bambang Riyanto Trilaksono · Rahman Indra Kesuma, S.Kom., M.Cs.

**Source** Chollet & Watson, Deep Learning with Python 3e — chapter 13

LEARNING OUTCOMES

# Session Objectives

By the end of this session, participants can:

1. Name the four common timeseries tasks and say which are supervised.
2. Build a **common-sense baseline** for a forecasting problem and evaluate it properly.
3. Explain **why a dense network and a 1D ConvNet fail** on this problem — and what that says about architecture priors.
4. Explain what an **RNN** is, and how it differs from a feedforward network.
5. Say why `SimpleRNN` **is not used in practice**, and how **LSTM** solves it.
6. Apply **recurrent dropout** correctly, and say why the naive version is harmful.
7. **Stack** recurrent layers with `return_sequences`, and know when a **bidirectional** layer helps and when it cannot.

BOOK
Chapter 13 — full text

NOTEBOOK
01_jena_data_and_baseline.ipynb

NOTEBOOK
02_dense_and_conv1d.ipynb

NOTEBOOK
03_lstm_and_gru.ipynb

NOTEBOOK
04_dropout_stacking_bidirectional.ipynb

NOTEBOOK
All chapter 13 notebooks

REPO
Author's official notebook

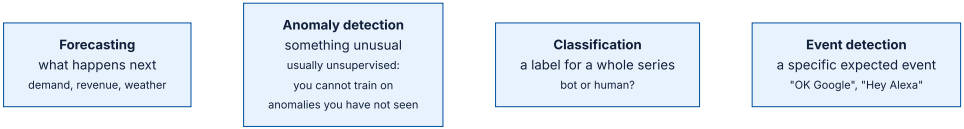
SECTION 13.1

# What a timeseries is, and why it is different

Unlike everything so far, working with timeseries means understanding **the dynamics of a system** — its periodic cycles, how it trends, its regular regime, and **its sudden spikes**

SECTION 13.1

# Four things you can do with one




Forecasting is by far the most common, and is what this chapter covers.

Note the parenthetical on anomaly detection: it is usually **unsupervised**, because you often do not know what kind of anomaly you are looking for — so **you cannot train on examples of it**

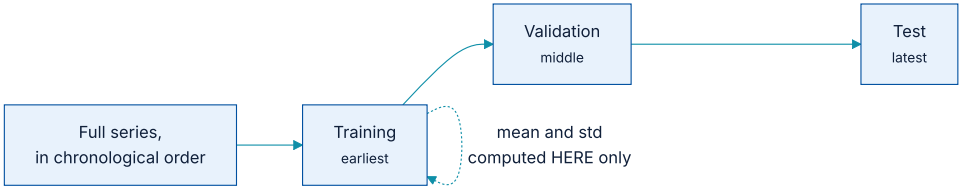
SECTION 13.2

# The running example

The data is the Jena weather dataset — fourteen meteorological quantities recorded every ten minutes over several years. It is small enough to work with and **genuinely hard enough to be instructive**

SECTION 13.2.1

# The split has to respect time




Chapter 5's arrow-of-time pitfall, made concrete.

The data must **not** be shuffled before splitting: all test data has to be later than all training data. And the normalisation statistics come from **the training portion only**

# Normalising fourteen quantities on different scales

listing 13.6 — normalising PYTHON

```
mean = raw_data[:num_train_samples].mean(axis=0)
raw_data -= mean
std = raw_data[:num_train_samples].std(axis=0)
raw_data /= std
```

Note the slice: `[:num_train_samples]`. The statistics are computed on the **first 210,225 timesteps only** — **the same information-leak rule as chapter 4's housing example**

SECTION 13.2.1

# Windows without copying the data


Consecutive samples share most of their timesteps, so materialising each one would waste enormous memory.

Keras has a utility for exactly this — `timeseries_dataset_from_array()` — which **generates the windows on the fly**, keeping only the original array in memory.

# Understanding it on a toy example first

the utility, on integers

PYTHON

```python
int_sequence = np.arange(10)

dummy_dataset = keras.utils.timeseries_dataset_from_array(
    data=int_sequence[:-3],       # windows come from here
    targets=int_sequence[3:],     # the target is 3 steps ahead
    sequence_length=3,
    batch_size=2,
)

for inputs, targets in dummy_dataset:
    for i in range(inputs.shape[0]):
        print([int(x) for x in inputs[i]], int(targets[i]))
```

OUTPUT

```
[0, 1, 2] 3
[1, 2, 3] 4
[2, 3, 4] 5
[3, 4, 5] 6
[4, 5, 6] 7
```

SECTION 13.2.1

# …then the real thing

the training dataset

PYTHON

```python
sampling_rate = 6        # one sample per hour (data is every 10 minutes)
sequence_length = 120    # five days of hourly readings
delay = sampling_rate * (sequence_length + 24 - 1)

train_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets=temperature[delay:],
    sampling_rate=sampling_rate,
    sequence_length=sequence_length,
    shuffle=True,
    batch_size=256,
    start_index=0,
    end_index=num_train_samples,
)
```

`start_index` and `end_index` are how the three splits are carved out of one array **without shuffling** — **the chronological order is preserved by construction**

01

# The baseline, and two failures

Common sense contains information a model has no access to.

SECTION 13.2.2

# Before any model: what would common sense do?

listing 13.9 — the common-sense baseline

PYTHON

```python
def evaluate_naive_method(dataset):
    total_abs_err, samples_seen = 0.0, 0
    for samples, targets in dataset:
        # column 1 is temperature; un-normalise it back to degrees Celsius
        preds = samples[:, -1, 1] * std[1] + mean[1]
        total_abs_err += np.sum(np.abs(preds - targets))
        samples_seen += samples.shape[0]
    return total_abs_err / samples_seen

print(f"Validation MAE: {evaluate_naive_method(val_dataset):.2f}")
print(f"Test MAE: {evaluate_naive_method(test_dataset):.2f}")
```

OUTPUT

```
Validation MAE: 2.44
Test MAE: 2.62
```

SECTION 13.2.2

# That is the number to beat

> Assume tomorrow is like today and you are off by **two and a half degrees on average**. Not good enough to launch a forecasting service — but **every model from here on has to beat it to justify itself**

The book makes the general point too: for an unbalanced classification task with 90% class A, *always predict A* scores 90%. **Such elementary baselines can prove surprisingly hard to beat.**

# Attempt 1 — a small dense network

listing 13.10 — a densely connected model PYTHON

```python
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.Flatten()(inputs)
x = layers.Dense(16, activation="relu")(x)
outputs = layers.Dense(1)(x)            # no activation: a regression output
model = keras.Model(inputs, outputs)

model.compile(optimizer="adam", loss="mse", metrics=["mae"])
history = model.fit(train_dataset, epochs=10,
                    validation_data=val_dataset, callbacks=callbacks)
```

**MSE as the loss, MAE as the metric.** MSE is used for training because it is **smooth around zero**, which gradient descent needs; MAE is reported because it is the number a human can interpret.

SECTION 13.2.3

# . . . and it does not beat the baseline

> Some validation losses come close to the no-learning baseline, **but not reliably**. This is exactly why the baseline was worth computing: it turns out to be **not easy to outperform**

As the book puts it: **your common sense contains a lot of valuable information to which a machine learning model has no access.**

SECTION 13.2.3

# Why gradient descent could not find the obvious answer

> Because your **hypothesis space** is the space of all two-layer networks with that configuration, and the heuristic is **one model among millions** in it. **Like looking for a needle in a haystack.**

The general limitation: **unless the learning algorithm is hardcoded to look for a specific kind of simple model, it can fail to find a simple solution to a simple problem.**

SECTION 13.2.4

# Attempt 2 — a 1D ConvNet

the 1D convolutional model

PYTHON

```
x = layers.Conv1D(8, 24, activation="relu")(inputs)
x = layers.MaxPooling1D(2)(x)
x = layers.Conv1D(8, 12, activation="relu")(x)
x = layers.MaxPooling1D(2)(x)
x = layers.Conv1D(8, 6, activation="relu")(x)
x = layers.GlobalAveragePooling1D()(x)
outputs = layers.Dense(1)(x)
```

OUTPUT

```
validation MAE: about 2.9 degrees
```

SECTION 13.2.4

# Worse than the dense model. Two reasons.

**Weather is not translation invariant**

There are daily cycles, but **morning data behaves differently from evening or night data**. The invariance holds only at one very specific timescale.

**Order matters, a lot**

The recent past is far more informative than five days ago — and **max pooling and global average pooling destroy order information**.

A worked demonstration of chapter 9's claim: an architecture encodes assumptions, and **when those assumptions are false the architecture actively hurts**

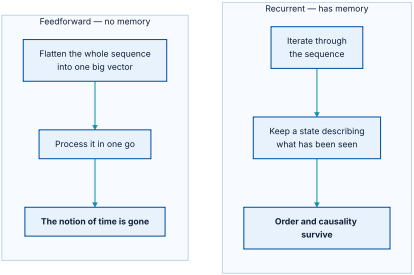
# 02

# Recurrent neural networks

Treat the data as what it is: a sequence where order matters.

SECTION 13.3.1

# Every network so far had no memory




The dense model removed time; the convolutional model destroyed order. Neither treated the data as a sequence.

SECTION 13.3.1

# The biological analogy the book uses

Biological intelligence processes information **incrementally, while maintaining an internal model** built from past information and constantly updated. An RNN adopts the same principle — **in an extremely simplified version**

SECTION 13.3.1

# An RNN is a network with an internal loop

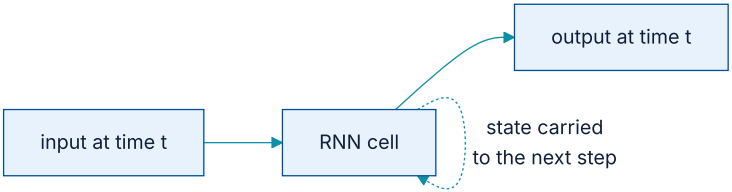



Figure 13.6 — it iterates through the sequence, keeping a state describing what it has seen so far.

The state is **reset between independent sequences**, so one sequence is still one data point. What changes is that **the data point is no longer processed in a single step**

SECTION 13.3.2

# Why you will rarely use SimpleRNN

> The cause is **the vanishing gradient problem** — the same effect chapter 9 described for very deep feedforward networks. Studied by Hochreiter, Schmidhuber and Bengio in the early 1990s.

Keras offers two alternatives designed to fix it: `LSTM` and `GRU`.

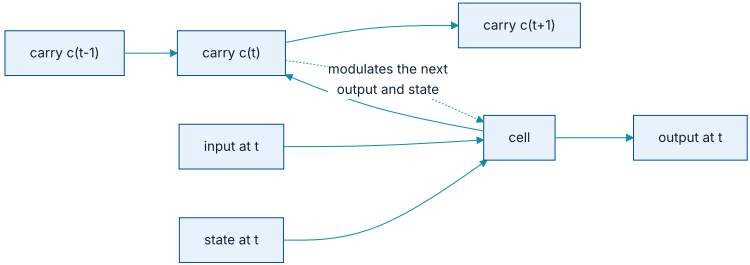
SECTION 13.3.2

# LSTM: a conveyor belt running beside the sequence

> Imagine a **conveyor belt running parallel to the sequence**. Information can jump onto it at any point, be carried to a later timestep, and jump off **intact** when needed — **preventing older signals from gradually vanishing**

The book points out this should remind you of **residual connections** from chapter 9. It is very nearly the same idea, applied along time rather than along depth.

# The carry track, in a picture




Figures 13.8–13.9 — the carry `c` modulates both the next output and the next state.

That is all the structural difference amounts to: a `SimpleRNN` cell, plus **an extra data flow that crosses timesteps without being squashed at every step**.

SECTION 13.3.2

# And it finally beats the baseline

a simple LSTM model

PYTHON

```python
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.LSTM(16)(inputs)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

model.compile(optimizer="adam", loss="mse", metrics=["mae"])
history = model.fit(train_dataset, epochs=10,
                    validation_data=val_dataset, callbacks=callbacks)
```

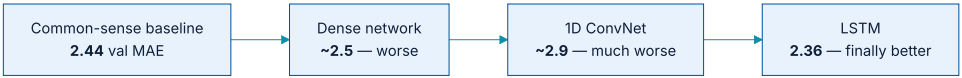
**2.44**

common-sense baseline, validation MAE

**2.36**

LSTM, validation MAE

# The whole chapter, as one ladder




Two deep learning models lose to the heuristic before one beats it.

> The margin over the baseline is **small**, and the chapter does not pretend otherwise. The lesson is not that LSTMs are powerful; it is that **the right architecture prior is what made the difference**

# 03

# Getting more out of RNNs

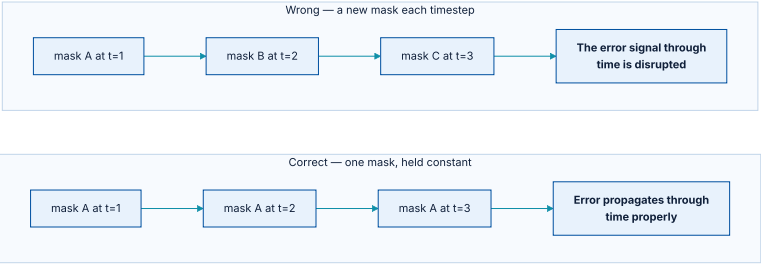
Recurrent dropout, stacking, and bidirectional layers.

SECTION 13.3.4

# Dropout in a recurrent layer is not obvious

It has long been known that applying dropout **before** a recurrent layer **hinders learning rather than helping regularisation**

# Yarin Gal's result: hold the mask constant




The same pattern of dropped units at every timestep, rather than a fresh random one.

Determined in 2015 as part of Gal's PhD thesis on Bayesian deep learning. He did the research **using Keras**, and helped build the mechanism directly into its recurrent layers.

SECTION 13.3.4

# The two arguments that implement it

recurrent dropout PYTHON

```python
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.LSTM(32, recurrent_dropout=0.25)(inputs)
x = layers.Dropout(0.5)(x)              # regularise the LSTM's OUTPUT too
outputs = layers.Dense(1)(x)
```

| ARGUMENT | WHAT IT DROPS |
| --- | --- |
| `dropout` | The layer's **input** units. |
| `recurrent_dropout` | The layer's **inner recurrent activations** — a temporally constant mask, which is the part Gal's result is about. |

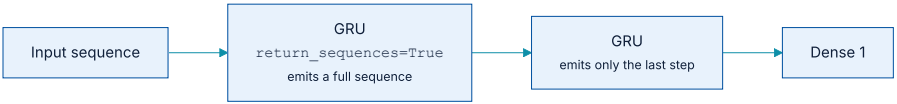
SECTION 13.3.5

# Stacking: when you stop overfitting, add capacity

Recurrent layer stacking is the classic way to do it. Not long ago **Google Translate was powered by a stack of seven large LSTM layers** — **which is huge**

SECTION 13.3.5

# The one argument that makes stacking work




Intermediate layers must emit a full sequence; only the last one emits a single vector.

a stack of two GRU layers — PYTHON

```
x = layers.GRU(32, recurrent_dropout=0.5, return_sequences=True)(inputs)
x = layers.GRU(32, recurrent_dropout=0.5)(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1)(x)
```

`return_sequences=True` makes a layer return a **rank-3 tensor** — its output at every timestep — instead of only the last one. Omit it on an intermediate layer and **the next layer has no sequence to read**

SECTION 13.3.5

# GRU, and why the book switches to it here

In practice the choice between them is empirical. GRU is **cheaper to run**; LSTM has **slightly more representational power**. **Try both and measure** rather than arguing about it.

SECTION 13.3.6

# Bidirectional layers: the same data, read backwards too

wrapping a layer

PYTHON

```
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.Bidirectional(layers.LSTM(16))(inputs)
outputs = layers.Dense(1)(x)
```

On **this** problem it does not help: for weather, the recent past is what matters, so reading in reverse gives a **chronologically backwards view that is simply worse**. It shines on text, where both directions carry meaning — chapter 14 returns to it.

SECTION 13.4

# Going further

- **Adjust the number of units** in each recurrent layer, and the dropout rates — the configurations here were chosen largely arbitrarily.
- **Adjust the learning rate** of the optimizer.
- Try a **stack of `Dense` layers** as the regressor on top instead of a single one.
- **Improve the input**: try longer or shorter sequences, or a different sampling rate.

> And the standing warning: **always evaluate on the validation set**, then confirm once on test — otherwise you end up **overfitting to your validation procedure**, exactly as chapter 5 described.

PRACTICE

# What this chapter changes about how you start

1. **Compute the baseline before writing any model.** It is cheap, and here it would have saved two failed attempts.
2. **State the architecture prior out loud.** *Convolution assumes translation invariance* — is that true of your data, at the timescale you care about?
3. **Check whether order matters.** If it does, any layer that pools or flattens is throwing away the signal.

> For transactional or operational data these three questions usually settle the architecture before a single experiment runs — which is **the cheapest hour in the whole project**

SUMMARY

# What has to survive this chapter

1. **Always establish a common-sense baseline first.** Here it beat two deep learning models.
2. **A good solution existing in your hypothesis space does not mean gradient descent will find it.**
3. **Dense flattens away time; 1D convolution pools away order.** Neither prior fits a forecasting problem.
4. **RNNs keep a state** and process a sequence incrementally.
5. `SimpleRNN` **cannot learn long dependencies**; LSTM adds a carry track — residual connections, along time.

SUMMARY

# . . . and the three refinements

**Recurrent dropout**

A **temporally constant mask**. A fresh random mask each timestep disrupts the error signal through time.

**Stacking**

More capacity, at more compute. Intermediate layers need `return_sequences=True`.

**Bidirectional**

Helps where **both directions carry meaning** — text. Does not help where only the recent past matters.

NOTEBOOK

03_lstm_and_gru.ipynb

NEXT

Chapter 14 — Text classification